

Continuous Delivery (CD) for an ML Application

GitHub Actions • Docker • GHCR • Staging • Approval Gates • Rollback

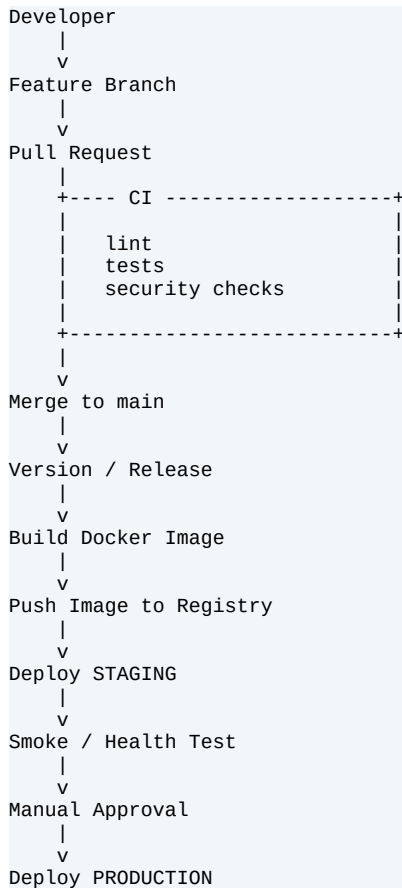
Duration	~1.5 hours
Level	Graduating CS / MLOps
Prerequisites	Git, commits/push, branches, PRs, semantic versioning, GitHub Actions CI, basic Docker
Outcome	A versioned Docker artifact automatically delivered to staging and promoted to production after approval

Lecture focus: move beyond CI into controlled, repeatable delivery of tested artifacts.

Learning Objectives

- Differentiate Continuous Integration, Continuous Delivery, and Continuous Deployment.
- Explain build-once/deploy-many and immutable artifact principles.
- Package an ML inference API as a Docker image.
- Tag and publish images using semantic application versions.
- Use GitHub Environments for staging and production.
- Store deployment credentials using GitHub Secrets.
- Automatically deploy to staging and validate with a health/smoke test.
- Require a manual approval gate before production deployment.
- Perform a basic rollback to a previously known-good version.
- Relate software delivery to model, dataset, and ML artifact versioning.

1. Start with the Delivery Pipeline



Teaching note: Ask: "Where does CI finish and CD begin?" A useful answer is: CI establishes that the code is good enough to integrate; CD establishes that the resulting artifact is good enough to release.

2. Continuous Delivery vs Continuous Deployment

Continuous Integration:

Code -> Build -> Test

Continuous Delivery:

Code -> Build -> Test -> Package -> Stage -> Ready for Production
|
Human Approval

Continuous Deployment:

Code -> Build -> Test -> Package -> Stage -> Production

The important distinction is the production gate. In this tutorial, staging is automatic while production requires approval.

3. Create the Demo ML Application

```
mlops-cd-demo/  
|  
+-- app.py  
+-- requirements.txt
```

```

+-- Dockerfile
+-- compose.yaml
+-- VERSION
+-- tests/
|   +-- test_app.py
+-- .github/
|   +-- workflows/
|       +-- cd.yml
+
mkdir mlops-cd-demo
cd mlops-cd-demo

git init

```

4. Build a Tiny Inference API

Create app.py:

```

from flask import Flask, jsonify, request

app = Flask(__name__)

MODEL_VERSION = "1.0"

@app.route("/")
def home():
    return jsonify({
        "service": "mlops-demo",
        "status": "running"
    })

@app.route("/health")
def health():
    return jsonify({
        "status": "healthy",
        "model_version": MODEL_VERSION
    })

@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()
    value = float(data["value"])

    # Dummy ML prediction for teaching
    prediction = value * 2

    return jsonify({
        "input": value,
        "prediction": prediction,
        "model_version": MODEL_VERSION
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

```

requirements.txt:

```

flask==3.1.2
pytest==8.4.2

python -m venv .venv
source .venv/bin/activate

pip install -r requirements.txt
python app.py

```

Test the health endpoint:

```

curl http://localhost:5000/health
{
  "model_version": "1.0",
  "status": "healthy"
}

```

5. Add Tests

tests/test_app.py:

```

from app import app

def test_health():
    client = app.test_client()
    response = client.get("/health")

    assert response.status_code == 200
    data = response.get_json()
    assert data["status"] == "healthy"

```

```
def test_prediction():
    client = app.test_client()
    response = client.post("/predict", json={"value": 5})

    assert response.status_code == 200
    data = response.get_json()
    assert data["prediction"] == 10

pytest
```

Teaching note: Students already know CI, so move quickly. Tests are not the CD pipeline; they are one of the gates that allow an artifact to enter the delivery pipeline.

6. Containerize the Application

Dockerfile:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]

docker build -t mlops-cd-demo:local .

docker run --rm -p 5000:5000 mlops-cd-demo:local

curl http://localhost:5000/health
```

7. Immutable Artifacts: Build Once, Deploy Many

Avoid rebuilding separately for staging and production:

DO NOT:

```
Build staging image
Build production image
```

Instead:

```
BUILD ONCE

mlops-cd-demo:1.2.0

|
+-----> STAGING
|
+-----> PRODUCTION
```

Teaching note: This is one of the central CD ideas. The exact artifact tested in staging should be promoted to production. If tested artifact != deployed artifact, the testing guarantee is weaker.

8. Version the Application

Create VERSION:

```
1.0.0
git add .
git commit -m "feat: add inference service"
```

Connect semantic versions to container artifacts:

Git tag	Docker image
v1.0.0	-> ghcr.io/org/app:1.0.0
v1.1.0	-> ghcr.io/org/app:1.1.0
v2.0.0	-> ghcr.io/org/app:2.0.0

9. Introduce the Container Registry

Use GitHub Container Registry (GHCR):

```
GitHub Repository
|
v
GitHub Actions
|
```

```

      v
docker build
      |
      v
GHCR
  |
  v
ghcr.io/<username>/mlops-cd-demo:1.0.0

```

A registry answers a fundamental delivery question: where does the deployable artifact live?

10. Start the CD Workflow

Create `.github/workflows/cd.yml`:

```

name: Continuous Delivery

on:
  push:
  tags:
    - "v*.*.*"

permissions:
  contents: read
  packages: write

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Run tests
        run: pytest

```

Teaching note: The trigger is a semantic version tag, not every normal commit. A tag such as `v1.0.0` represents a releasable version.

11. Extract the Version Number

```

build:
  needs: test
  runs-on: ubuntu-latest

  outputs:
    version: ${ steps.version.outputs.version }

  steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Get version
      id: version
      run: |
        VERSION=${GITHUB_REF_NAME#v}
        echo "version=$VERSION" >> "$GITHUB_OUTPUT"

    - name: Show version
      run: |
        echo "Building version ${ steps.version.outputs.version }"

```

If `GITHUB_REF_NAME` is `v1.3.0`, `${GITHUB_REF_NAME#v}` produces `1.3.0`.

12. Login to GitHub Container Registry

```

- name: Login to GHCR
  uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${ github.actor }
    password: ${ secrets.GITHUB_TOKEN }

```

`GITHUB_TOKEN` is generated automatically for the workflow. `packages: write` allows the job to publish container packages.

13. Build and Push the Image

```
- name: Build Docker image
  run: |
    docker build \
      -t ghcr.io/${{ github.repository }}:${{ steps.version.outputs.version }} \
      .

- name: Push Docker image
  run: |
    docker push \
      ghcr.io/${{ github.repository }}:${{ steps.version.outputs.version }}

git tag v1.0.0
git push origin v1.0.0
```

This produces an artifact such as `ghcr.io/username/mlops-cd-demo:1.0.0`.

14. Add a Convenience latest Tag

```
- name: Build Docker image
  run: |
    docker build \
      -t ghcr.io/${{ github.repository }}:${{ steps.version.outputs.version }} \
      -t ghcr.io/${{ github.repository }}:latest \
      .

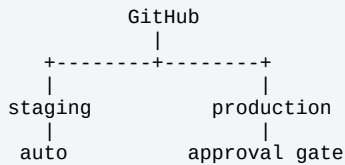
- name: Push Docker images
  run: |
    docker push ghcr.io/${{ github.repository }}:${{ steps.version.outputs.version }}
    docker push ghcr.io/${{ github.repository }}:latest
```

Use explicit version tags for production reproducibility. Treat latest as a movable convenience tag.

15. Introduce GitHub Environments

```
Repository
-> Settings
-> Environments
```

```
Create:
 staging
 production
```



- Environment-specific secrets
- Variables
- Permissions
- Approvals
- Deployment history

16. Create a Staging Server

Any Ubuntu VM/server is sufficient for this classroom exercise.

```
sudo apt update
sudo apt install docker.io -y

sudo systemctl enable docker
sudo systemctl start docker

docker --version
```

17. Prepare SSH Access

```
ssh-keygen -t ed25519 -f deploy_key

ssh-copy-id -i deploy_key.pub user@SERVER_IP

ssh -i deploy_key user@SERVER_IP
```

18. Store Deployment Secrets

For the staging environment, add:

```
STAGING_HOST
STAGING_USER
STAGING_SSH_KEY
```

Example:

```
STAGING_HOST=192.168.1.100
STAGING_USER=ubuntu
```

Teaching note: Never hard-code deployment passwords or private keys into YAML. Store deployment credentials in GitHub Secrets.

19. Automatic Staging Deployment

```
deploy-staging:
  needs: build
  runs-on: ubuntu-latest

  environment:
    name: staging

  steps:
    - name: Deploy to staging
      uses: appleboy/ssh-action@v1
      with:
        host: ${ secrets.STAGING_HOST }
        username: ${ secrets.STAGING_USER }
        key: ${ secrets.STAGING_SSH_KEY }

        script: |
          docker pull \
            ghcr.io/${ github.repository }:${ needs.build.outputs.version }

          docker stop mlops-api || true
          docker rm mlops-api || true

          docker run -d \
            --name mlops-api \
            --restart unless-stopped \
            -p 5000:5000 \
            ghcr.io/${ github.repository }:${ needs.build.outputs.version }

tag
|
v
test
|
v
build
|
v
registry
|
v
staging
```

20. Add a Smoke Test

A successful Docker command does not prove that the API responds correctly.

```
    - name: Verify staging deployment
      run: |
        sleep 5

        curl --fail \
          http://${ secrets.STAGING_HOST }:5000/health

Deploy
|
v
Health endpoint
|
+----- FAIL -----> Stop pipeline
|
SUCCESS
|
v
Candidate for production
```

21. Production Approval Gate

Create a production environment and configure Required reviewers under Repository -> Settings -> Environments -> production.

```
deploy-production:
  needs:
```

```

- build
- deploy-staging

runs-on: ubuntu-latest

environment:
  name: production

steps:
- name: Deploy production
  uses: appleboy/ssh-action@v1
  with:
    host: ${ secrets.PRODUCTION_HOST }
    username: ${ secrets.PRODUCTION_USER }
    key: ${ secrets.PRODUCTION_SSH_KEY }

    script: |
      docker pull \
        ghcr.io/${ github.repository }:${ needs.build.outputs.version }

      docker stop mlops-api || true
      docker rm mlops-api || true

      docker run -d \
        --name mlops-api \
        --restart unless-stopped \
        -p 5000:5000 \
        ghcr.io/${ github.repository }:${ needs.build.outputs.version }

```

The workflow pauses at “Waiting for approval.” Once an authorized reviewer approves it, production deployment starts. That is Continuous Delivery.

22. Complete Workflow

```

name: Continuous Delivery

on:
  push:
    tags:
      - "v*.*.*"

permissions:
  contents: read
  packages: write

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Test
        run: pytest

  build:
    needs: test
    runs-on: ubuntu-latest

    outputs:
      version: ${ steps.version.outputs.version }

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Get version
        id: version
        run: |
          VERSION=${GITHUB_REF_NAME#v}
          echo "version=$VERSION" >> "$GITHUB_OUTPUT"

      - name: Login GHCR
        uses: docker/login-action@v3

```



```

    with:
      registry: ghcr.io
      username: ${GITHUB_ACTOR}
      password: ${GITHUB_TOKEN}

- name: Build
  run: |
    docker build \
      -t ghcr.io/${GITHUB_REPOSITORY}:${GITHUB_OUTPUT_VERSION} \
      -t ghcr.io/${GITHUB_REPOSITORY}:latest \
      .

- name: Push
  run: |
    docker push ghcr.io/${GITHUB_REPOSITORY}:${GITHUB_OUTPUT_VERSION}
    docker push ghcr.io/${GITHUB_REPOSITORY}:latest

deploy-staging:
  needs: build
  runs-on: ubuntu-latest

  environment:
    name: staging

  steps:
    - name: Deploy staging
      uses: appleboy/ssh-action@v1
      with:
        host: ${STAGING_HOST}
        username: ${STAGING_USER}
        key: ${STAGING_SSH_KEY}

        script: |
          docker pull \
            ghcr.io/${GITHUB_REPOSITORY}:${GITHUB_OUTPUT_VERSION}

          docker stop mlops-api || true
          docker rm mlops-api || true

          docker run -d \
            --name mlops-api \
            --restart unless-stopped \
            -p 5000:5000 \
            ghcr.io/${GITHUB_REPOSITORY}:${GITHUB_OUTPUT_VERSION}

    - name: Smoke test
      run: |
        sleep 5
        curl --fail \
          http://${STAGING_HOST}:5000/health

deploy-production:
  needs:
    - build
    - deploy-staging
  runs-on: ubuntu-latest

  environment:
    name: production

  steps:
    - name: Deploy production
      uses: appleboy/ssh-action@v1
      with:
        host: ${PRODUCTION_HOST}
        username: ${PRODUCTION_USER}
        key: ${PRODUCTION_SSH_KEY}

        script: |
          docker pull \
            ghcr.io/${GITHUB_REPOSITORY}:${GITHUB_OUTPUT_VERSION}

          docker stop mlops-api || true
          docker rm mlops-api || true

          docker run -d \
            --name mlops-api \
            --restart unless-stopped \
            -p 5000:5000 \
            ghcr.io/${GITHUB_REPOSITORY}:${GITHUB_OUTPUT_VERSION}

```

23. Release Version 1.0.0

```
git checkout main
git pull

git tag v1.0.0
git push origin v1.0.0
```

Students should watch GitHub -> Actions -> Continuous Delivery.

```
✓ test
✓ build
✓ deploy-staging
🔴 deploy-production
  Waiting for approval

After approval:
✓ deploy-production
```

24. Demonstrate a New Release

Change `MODEL_VERSION = "1.0"` to `MODEL_VERSION = "1.1"`, then use the normal branch -> commit -> push -> PR -> CI -> review -> merge flow.

```
git checkout main
git pull

git tag v1.1.0
git push origin v1.1.0

Registry:
1.0.0
1.1.0
latest

Production promotion:
1.0.0 -> 1.1.0
```

25. Rollback

Scenario: 1.1.0 has a serious prediction problem. Because earlier images are versioned and immutable, rollback is simple.

```
1.0.0 GOOD
1.1.0 BAD

docker stop mlops-api
docker rm mlops-api

docker run -d \
  --name mlops-api \
  --restart unless-stopped \
  -p 5000:5000 \
  ghcr.io/USERNAME/mlops-cd-demo:1.0.0

curl http://SERVER_IP:5000/health
```

Teaching note: This is why relying only on latest is insufficient. Explicit version tags let you reproduce and restore a known deployment.

26. Teach Promotion, Not Rebuilding

Bad approach:

```
main
|
build staging
|
test staging
|
rebuild production
|
production
```

Better approach:

```
main
|
BUILD 1.1.0
|
Registry
|
1.1.0
+---- staging
|
+---- production
```

The artifact moves through environments; the source code is not rebuilt at every stage.

27. Connect CD with MLOps

For normal software:

```
Source Code
  |
  v
Docker image
  |
  v
Deployment
```

For ML:

```
Code
+
Model
+
Dependencies
+
Preprocessing
+
Configuration
  |
  v
Reproducible artifact
  |
  v
Deployment
```

Example ML artifact:

```
ml-api:1.4.0

contains:
Flask API
scikit-learn runtime
preprocessing.py
model.pkl
feature schema
runtime dependencies
```

MLOps delivery is more complex because software and model versions can change independently.

28. Model/Application Versioning

```
Application Version: 2.3.0
Model Version:      fraud-model-17
Dataset Version:    transactions-v31

Deployment
+-- app      = 2.3.0
+-- model    = 17
+-- dataset  = 31
+-- image    = sha256:abcd...
```

Teaching note: Ask: "If prediction accuracy deteriorates, what exactly should we rollback?" This prepares students for DVC, MLflow Model Registry, model promotion, and monitoring.

29. Recommended Classroom Timing

Time	Activity
0-10 min	CI vs CD vs Continuous Deployment
10-20 min	Flask ML API + tests
20-30 min	Docker artifact
30-40 min	Registry + immutable artifacts
40-55 min	GitHub Actions build/push
55-65 min	Staging environment
65-75 min	Health check + production approval
75-82 min	Release v1.0.0 -> v1.1.0
82-87 min	Rollback
87-90 min	MLOps discussion

30. Student Exercise: Build a Professional CD Pipeline

Students should complete the following independently:

- PRs continue to run CI only.
- A semantic version tag such as v1.3.0 starts CD.

- Publish ghcr.io/student/ml-api:1.3.0 and ghcr.io/student/ml-api:latest.
- Deploy to staging automatically.
- Require GET /health to pass before production becomes available.
- Require manual approval for production.
- Return application_version, model_version, and status from /health.
- Demonstrate rollback from 1.3.0 to 1.2.0.

Target /health response:

```
{
  "application_version": "1.3.0",
  "model_version": "model-7",
  "status": "healthy"
}
```

Bonus Exercise: Production Traceability

Capture the Git commit:

```
git rev-parse --short HEAD
```

Expose it through /health:

```
{
  "application_version": "1.3.0",
  "model_version": "model-7",
  "git_commit": "f72ab81",
  "status": "healthy"
}
```

```
Production
|
Container 1.3.0
|
Git commit f72ab81
|
Pull Request #27
```

Recommended Next Lecture

Production Deployment Strategies: Rolling, Recreate, Blue/Green, and Canary Releases.

Suggested course progression:

1. Git
2. Branching / PR
3. Semantic Versioning
4. CI with GitHub Actions
5. Continuous Delivery <- this tutorial
6. Docker Compose deployment
7. Deployment strategies
 - Rolling
 - Recreate
 - Blue/Green
 - Canary
8. Kubernetes
9. Kubernetes-based CD
10. GitOps with Argo CD
11. ML model delivery
 - DVC
 - MLflow Registry
 - model promotion
12. Monitoring + rollback

Teaching bridge: students first learn how to deliver version 1.1.0, then learn how to replace version 1.0.0 safely with controlled risk and minimal downtime.